SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3) **Total Marks: 30**
Question Count: 3

Code of Conduct

I **_Lavanya.M(192321051)_** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: __Lavanya.M__

Assessment Tasks

Case Study

1. **Case Study 1: Smart Retail Inventory Management System**

   A retail chain implements a cloud-based inventory system connected to store devices.Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. **Case Study 2: Cloud-Based Document Collaboration System**

   An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. **Case Study 3: Online Food Ordering Platform**

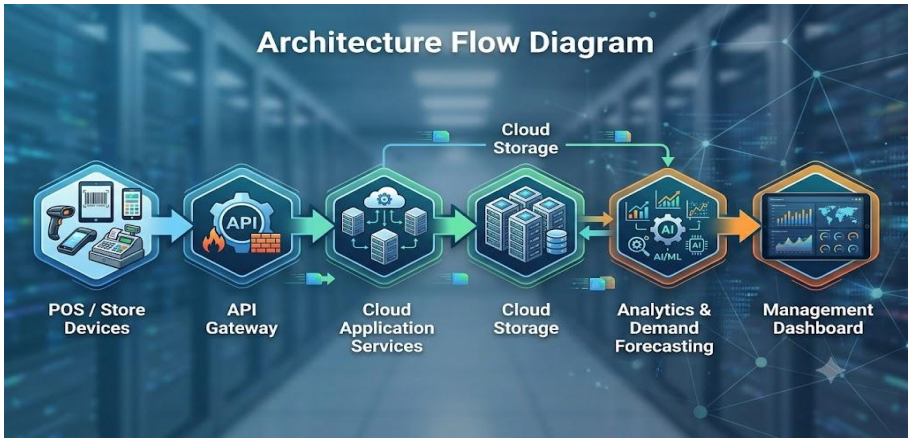
A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients. Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform. Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study 1: Smart Retail Inventory Management System

1. Cloud Architecture

The Smart Retail Inventory Management System uses a **cloud-based architecture** to connect Point-of-Sale (POS) systems, store devices, dashboards, cloud storage, and analytics services.

Architecture flow:



2. Implementation of Cloud Components

Component	Implementation
Clients	POS terminals, mobile devices and web dashboards
Web APIs	REST APIs for product updates, sales transactions and inventory queries
API Gateway	Receives API requests and manages authentication, routing and rate limiting
Cloud Storage	Stores product information, transaction logs, invoices and analytics datasets
Platform Services	Cloud-based analytics and machine-learning services for demand forecasting
Compute	Cloud VMs or containers execute inventory and business applications
Network Security	VPNs, firewalls and secure HTTPS communication
Database	Managed cloud database stores structured product and transaction information

3. Working

1. A customer purchases a product at a store.
2. The POS system sends the transaction through a **REST API**.
3. The API Gateway authenticates and routes the request to the cloud application.

4. The inventory database is updated automatically.
5. Transaction logs are stored in **cloud storage**.
6. Analytics services process historical sales data.
7. A demand forecasting service predicts future product requirements.
8. The dashboard displays stock levels and predicted demand to managers.
9. When inventory falls below a threshold, the system can generate a **reorder alert**.

4. Security

Security can be implemented using:

- **VPNs** for secure communication between stores and cloud infrastructure.
- **Firewalls** to control network traffic.
- **API Gateway authentication** to protect APIs.
- **HTTPS/TLS encryption** for data transmission.
- **Role-Based Access Control (RBAC)** for administrators, managers and employees.
- Encryption of sensitive data stored in the cloud.

5. Benefits

- Real-time inventory monitoring.
- Centralized storage of retail data.
- Reduced stock-outs and overstocking.
- Scalable cloud infrastructure.
- Data-driven demand forecasting.
- Secure communication between stores and cloud services.
- Reduced need for maintaining physical servers.

6. Cloud Services Used

The system demonstrates **IaaS, PaaS and SaaS** together:

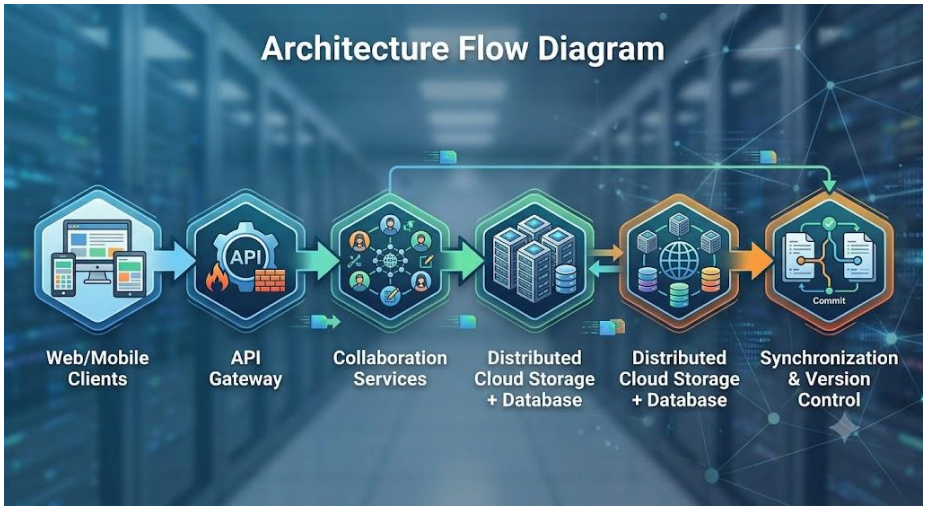
- **IaaS:** Virtual machines and networking infrastructure.
- **PaaS:** Analytics, databases and forecasting services.
- **SaaS:** Inventory management dashboard/application used by employees.

Case Study 2: Cloud-Based Document Collaboration System

1. Cloud Architecture

The document collaboration system provides browser-based access to documents and allows multiple users to edit and share documents in real time.

Architecture flow:



2. Implementation of Cloud Components

Component	Implementation
Clients	Web browsers, tablets and mobile applications
Web APIs	REST/WebSocket APIs for document operations and real-time synchronization
Cloud Storage	Stores documents and document versions
Database	Stores user, permission and document metadata
Collaboration Service	Handles simultaneous editing
Version Control	Maintains previous document versions
Identity Management	User authentication and authorization
Network Infrastructure	CDN and geographically distributed servers
Security	Encryption, access permissions and secure APIs

3. Working

1. A user logs into the collaboration platform through a browser.
2. The identity service authenticates the user.
3. The user opens a document stored in distributed cloud storage.
4. The document is retrieved through cloud APIs.
5. When the user edits the document, changes are sent to the collaboration server.
6. Real-time synchronization distributes the changes to other connected users.
7. The system maintains document versions so previous versions can be restored.
8. Access permissions determine whether users can **view, comment or edit** a document.
9. The latest document version is stored in cloud storage.

4. Real-Time Collaboration

Real-time collaboration can use **WebSockets**.

For example:

User A edits → WebSocket → Collaboration Server → WebSocket → Users B and C

This allows changes to appear on multiple devices with minimal delay.

For simultaneous editing, the platform can use techniques such as:

- **Operational Transformation (OT)**
- **Conflict-Free Replicated Data Types (CRDTs)**

These mechanisms help prevent users' changes from incorrectly overwriting one another.

5. Security

The system uses:

- **HTTPS/TLS** for secure communication.
- Encryption for stored documents.
- Identity management for user authentication.
- **RBAC** for document permissions.
- API authentication tokens.
- Audit logs to track document activities.

- Version history for recovery from accidental changes.

6. High Availability and Performance

The system can achieve high availability through:

- Multiple cloud servers.
- Load balancing.
- Distributed cloud storage.
- Database replication.
- Content Delivery Networks (CDNs).
- Auto-scaling based on user demand.

For example, if thousands of users access documents simultaneously, additional application instances can automatically be created.

7. Benefits

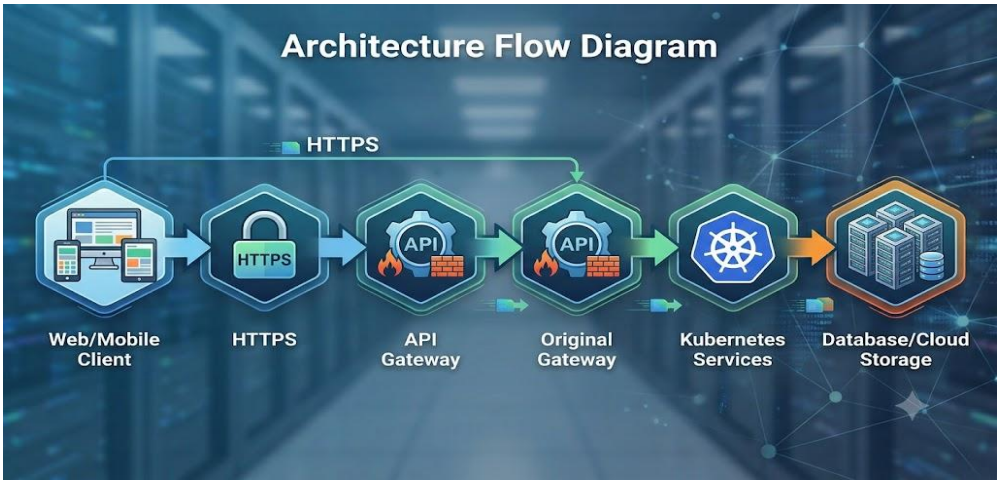
- Real-time collaboration from anywhere.
- Automatic document synchronization.
- Centralized document storage.
- Version history and recovery.
- Easy sharing and access control.
- High availability.
- Reduced dependency on local storage.

Case Study 3: Online Food Ordering Platform

1. Cloud Architecture

The Online Food Ordering Platform uses web and mobile clients connected to RESTful cloud APIs. Cloud storage, virtual machines and Kubernetes provide scalable backend infrastructure.

Architecture flow:



2. Implementation of Cloud Components

Component	**Implementation**
Clients	Web browsers and mobile applications
REST APIs	Handle login, menu, orders, payments and delivery requests
API Gateway	Routes and secures API requests
Cloud Platform	Hosts backend application services
Virtual Machines	Provide compute infrastructure
Kubernetes	Manages and scales containers
Cloud Storage	Stores menu images, invoices and files
Database	Stores customer, restaurant and order information
Authentication	Tokens/JWT
Security	HTTPS and RBAC

3. Working

1. A customer opens the food ordering website or mobile application.

2. The client sends an HTTPS request to the cloud API.
3. The API Gateway authenticates and routes the request.
4. The backend retrieves the restaurant and menu information from the database.
5. Menu images are retrieved from cloud storage.
6. The customer selects food items and places an order.
7. The order is sent through a REST API to the backend.
8. The order information is stored in the cloud database.
9. The restaurant receives the order and updates its status.
10. The customer receives real-time order status updates.
11. An invoice is generated and stored securely in cloud storage.

4. Role of Kubernetes

Kubernetes manages containerized backend services.

For example, the platform may have separate services for:

- User authentication
- Restaurant management
- Menu management
- Order processing
- Payment processing
- Notification service

If the number of customers increases during lunch time, Kubernetes can increase the number of running application containers.

Example:

5 containers → High traffic → 15 containers

When traffic decreases:

15 containers → Low traffic → 5 containers

This provides **elasticity** and avoids wasting resources.

5. Security

The platform implements:

- **HTTPS/TLS** to protect network communication.
- Authentication tokens such as JWT.
- **Role-Based Access Control** for customers, restaurant staff and administrators.
- API authentication and authorization.
- Encryption of stored customer information.
- Secure cloud storage permissions.
- Firewalls and network security controls.

6. Scalability and Reliability

The system can use:

- Load balancers.
- Kubernetes auto-scaling.
- Multiple virtual machines.
- Container replication.
- Database replication.
- Cloud storage.
- Monitoring and logging services.

If one application container fails, Kubernetes can automatically create another container, improving service availability.

7. Benefits

- Supports large numbers of simultaneous customers.
- Real-time order processing.
- Automatic scaling during peak hours.
- Secure customer and restaurant data.
- Reliable backend services.
- Centralized cloud storage.
- Easy deployment and maintenance.

Overall Comparison

Feature	Smart Retail	Document Collaboration	Food Ordering
Main Purpose	Inventory management	Real-time document editing	Online food ordering
Clients	POS, dashboards	Browser, mobile	Web, mobile
APIs	REST APIs	REST/WebSocket	REST APIs
Cloud Storage	Products, transactions, analytics	Documents and versions	Images, invoices, files
Platform Services	Analytics/forecasting	Collaboration/synchronization	Kubernetes/managed services
Security	VPN, firewall, API gateway	Encryption, IAM, permissions	HTTPS, tokens, RBAC
Scalability	Cloud scaling	Distributed infrastructure	Kubernetes auto-scaling
Key Cloud Benefit	Demand prediction	Real-time collaboration	Elastic order processing

Conclusion

All three case studies demonstrate how **cloud storage, web APIs and platform services** can be combined to build real-world cloud applications. APIs provide communication between clients and cloud services, cloud storage provides scalable data management, and platform services provide specialized capabilities such as analytics, real-time synchronization and container orchestration. Security mechanisms such as **encryption, authentication, API gateways and RBAC** ensure that these systems remain secure while scalability and high availability allow them to support changing workloads.